ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

ECYS | DITT
DESARROLLO DE TRANSFERENCIA TECNOLÓGICA

Día, Fecha:	Martes, 27/01/2026
Hora de inicio:	17:20

INTRODUCCIÓN A LA PROGRAMACIÓN Y COMPUTACIÓN 2

TOBÍAS RAFAEL ZAMORA SANTOS

Unidad 1: Introducción a .NET y C#

Introducción A La Programacion y Computacion 2

Escuela de Ingeniería de Ciencias Y Sistemas
Facultad de Ingeniería
Universidad de San Carlos de Guatemala

Agenda

RECORDATORIOS

¿Qué es .NET y C#?

Conceptos Clave de C#

Entorno de desarrollo

Visual Studio y Visual Studio Code

Recursos

SDK .NET

Visual Studio

Visual Studio Code

Competencias a Desarrollar

Comprensión del Ecosistema: Diferenciar claramente entre el lenguaje de programación C# y la plataforma .NET, entendiendo el rol de cada uno en el desarrollo de software.

Configuración del Entorno: Instalar y configurar correctamente el SDK de .NET y las herramientas de desarrollo necesarias para iniciar un proyecto.

Selección de Herramientas: Distinguir las diferencias, ventajas y casos de uso entre Visual Studio (IDE robusto) y Visual Studio Code (editor ligero) para elegir la herramienta adecuada.

Fundamentos Técnicos: Identificar los conceptos claves que hacen funcionar a las aplicaciones .NET (como la compilación y ejecución básica).

¿Qué es .NET?

.NET es una plataforma de desarrollo **gratuita, de código abierto (open-source) y multiplataforma** creada por Microsoft para construir muchos tipos diferentes de aplicaciones.

Hoy en día .NET corre nativamente en Linux, macOS y Windows.

Para poder crear software utilizando la plataforma .NET, es indispensable instalar el **.NET SDK (Software Development Kit)**. Este kit es el corazón del desarrollo: contiene el **compilador** (que traduce nuestro código C# a lenguaje intermedio), las **bibliotecas base** y la **CLI** (interfaz de línea de comandos). Es importante destacar que el SDK **ya incluye el Runtime**, por lo tanto, al instalar el SDK, obtenemos tanto la capacidad de *construir* aplicaciones como la capacidad de **ejecutarlas** en cualquier sistema operativo (Windows, Linux o macOS).

.NET

¿Qué es C#?

Hay que resaltar que **.NET** es **políglota**, al ser .NET la plataforma para desarrollar las aplicaciones, el .NET SDK el conjunto de herramientas para el desarrollo de estas, **.NET soporta varios lenguajes,** Los tres principales son **C# (el más popular)**, **F# (funcional**) y **Visual Basic (legacy/clásico)**.

C# (C Sharp) es un lenguaje de programación moderno, **orientado a objetos** y de **tipado seguro (type-safe)**. Fue creado por Microsoft para aprovechar al máximo la potencia de la plataforma .NET, permitiendo crear software robusto y eficiente para cualquier sistema operativo.

Sintaxis familiar: (Si saben C, C++ o Java, se sentirán en familiarizados, usa llaves {}, punto y coma ;, etc.).

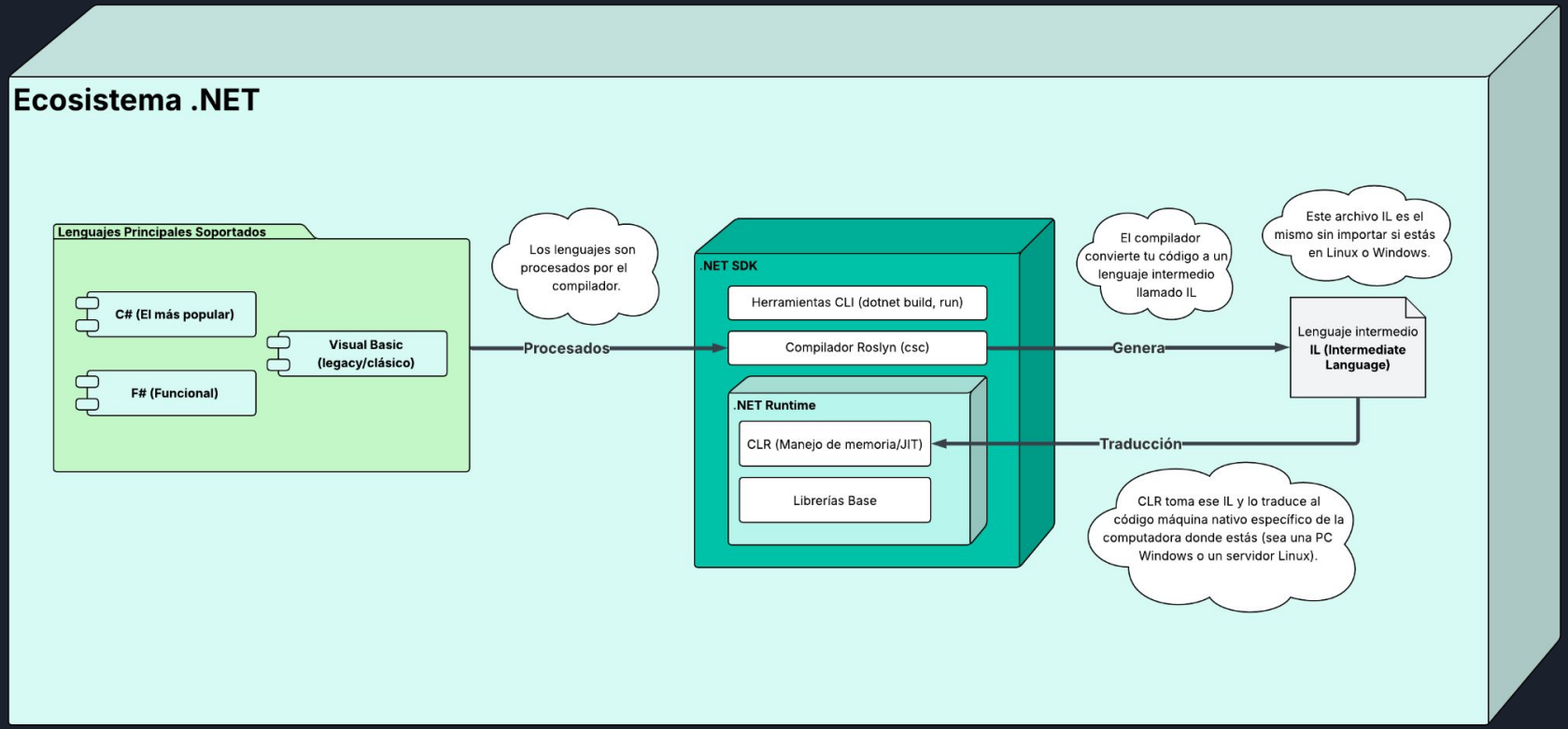
Orientado a Objetos: (Todo en C# se basa en clases y objetos, lo que facilita organizar proyectos grandes).

Similitud con Java y .NET

Ya que tuvieron la oportunidad de trabajar con **Java** en IPC1, si se pudieron dar cuenta el **JDK (Java Development Kit)** y el **JRE (Java Runtime Environment)**, serían lo que es el **.NET SDK (Software Development Kit)** y el **Runtime CLR** que ya viene dentro del .NET SDK .

Función	Ecosistema Java	Ecosistema .NET	¿Qué contiene?
Para Desarrollar (Crear apps)	JDK (Java Development Kit)	.NET SDK	Compilador (C#), Herramientas CLI y el Runtime
Para Ejecutar (Usar apps)	JRE (Java Runtime Environment)	.NET Runtime	Máquina Virtual (CLR) y Bibliotecas base

Ecosistema .NET



Analogías

Imagina que .NET es una cocina totalmente equipada, y C# es el chef experto que sabe cómo usar todos esos utensilios para preparar el platillo.

Si C# es el "coche", .NET es la "carretera, el motor y las leyes de tráfico". C# no hace mucho por sí solo sin las bibliotecas y el entorno de ejecución de .NET.

Conceptos Clave de C#

Orientado a Objetos (POO): Todo el código se organiza en Clases y Objetos para modelar la vida real.

"C# no nos deja escribir código 'suelto' o desordenado. Todo debe pertenecer a una clase. Esto nos ayuda a ser ordenados y a crear sistemas grandes y mantenibles, pensando en 'moldes' (clases) y 'piezas' (objetos)."

Tipado Estático y Seguro: El compilador valida los tipos de datos antes de ejecutar, evitando errores comunes.

"A diferencia de Python o JavaScript, C# es estricto. Si declaras una variable como número entero (int), no puedes meterle texto después. Esto parece rígido, pero es una gran ventaja: el programa te avisa del error antes de que lo ejecutes."

Case Sensitive: Distingue estrictamente entre mayúsculas y minúsculas (ej. Total ≠ total).

"Ojo aquí, porque es el error #1 del principiante. Para C#, la variable edad, Edad y EDAD son tres cosas totalmente diferentes. Si escriben Console.writeline con minúscula, fallará; debe ser Console.WriteLine."

Sintaxis Familia "C": Utiliza llaves { } para bloques y punto y coma ; al final de las instrucciones.

"Si alguno ya vio Java o C++ en la carrera, C# se les hará muy familiar. Usamos las llaves para agrupar código y siempre terminamos las sentencias con punto y coma. Es el estándar de la industria para lenguajes estructurados."

Sintaxis Básica de Una Clase en C#

A partir de C# 9, se introdujeron los **'Top-level statements'** (Instrucciones de nivel superior). Esto nos permite omitir la declaración explícita de la clase Program y el método Main

Ahora, el compilador trata al archivo **Program.cs** como el punto de entrada directo de la aplicación, generando automáticamente la estructura necesaria en segundo plano. Esto simplifica el código inicial y reduce el 'ruido visual' para comenzar a programar."

```
1  class MiClase
2  {
3      // Atributos o propiedades
4      public int miAtributo;
5
6      // Constructor
7      public MiClase(int valor)
8      {
9          miAtributo = valor;
10     }
11
12     // Métodos
13     public void MiMetodo()
14     {
15         Console.WriteLine("Este es un método de la clase MiClase.");
16     }
17 }
```

Entorno de Desarrollo

Es el conjunto de herramientas y programas que nos permiten escribir, compilar, probar y depurar nuestro código de manera eficiente. No es solo un editor de texto; es nuestro "taller de trabajo" digital.

Puntos clave:

- **Edición de Código:** Dónde escribimos la sintaxis (con colores y autocompletado).
- **Compilación:** La conexión con el SDK para crear el programa.
- **Depuración (Debugging):** Herramientas para encontrar y arreglar errores paso a paso.

Tipos Principales:

1. IDE (Entorno de Desarrollo Integrado):

- Solución "Todo en Uno".
- Incluye editor, compilador, depurador y herramientas visuales preinstaladas.
- *Ejemplo: Visual Studio.*

2. Editor de Código:

- Herramienta ligera y rápida.
- Se enfoca en la edición de texto y se expande mediante extensiones.
- *Ejemplo: Visual Studio Code.*

Visual Studio y Visual Studio Code

Visual Studio (IDE): Es como una cocina industrial de hotel. Tiene hornos gigantes, 20 tipos de sartenes y refrigeradores industriales. Tienes todo lo que podrías necesitar, pero ocupa mucho espacio y tardas en aprender a usarla.

Visual Studio Code (Editor): Es como una navaja suiza. Es pequeña, ligera y la llevas en el bolsillo. Si necesitas cortar, sacas la hoja; si necesitas abrir una botella, sacas el abridor. Tú le vas agregando accesorios (Extensiones) según lo que necesites en el momento.

En este curso usaremos .NET, que es tan flexible que nos permite trabajar cómodamente con cualquiera de los dos.

Recomendación: Visual Studio Code es excelente para empezar porque corre bien en cualquier máquina y es lo que más usarán si trabajan con tecnologías web o en Linux.

Nota importante: Para que VS Code funcione con C#, deben instalar la extensión **"C# Dev Kit"**. Sin ella, es solo un bloc de notas bonito.

Visual Studio vs Visual Studio Code

Característica	Visual Studio (IDE)	Visual Studio Code (Editor)
Tipo	IDE (Entorno de Desarrollo Integrado)	Editor de Texto Avanzado
Filosofía	"Todo incluido" Trae todo instalado de fábrica	"Hazlo tú mismo" Instalas solo lo que ocupas
Peso / Recursos	Pesado (requiere mucha RAM y espacio)	Ligero y rápido (funciona en casi cualquier PC)
Multiplataforma	Principalmente **Windows**	**Windows, Linux y Mac**
Uso ideal	Proyectos empresariales grandes y complejos	Web, scripts, proyectos rápidos y multiplataforma

Instalación de .NET SDK

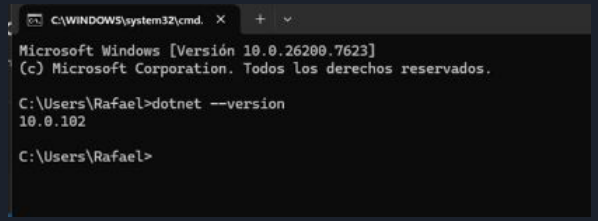
Sitio para descargar solo el .NET SDK 10.0

Descargar el .exe y instalar de forma normal.

Si la instalación se hizo de forma correcta al probar el comando:

dotnet —version

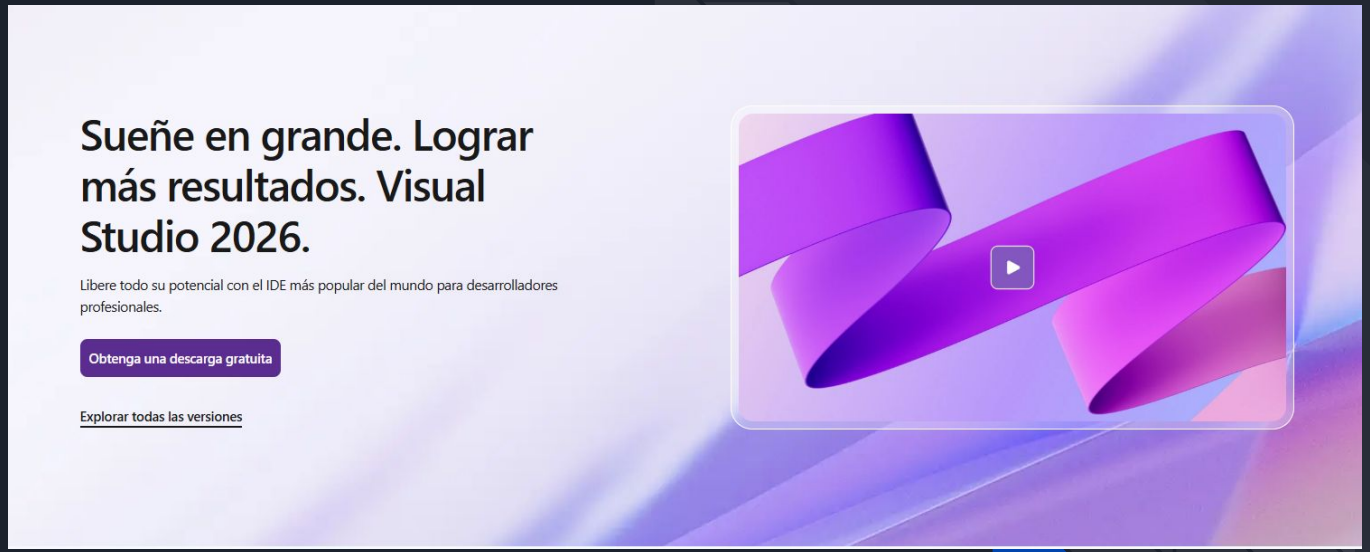
Debería de mostrar la versión instalada del SDK.



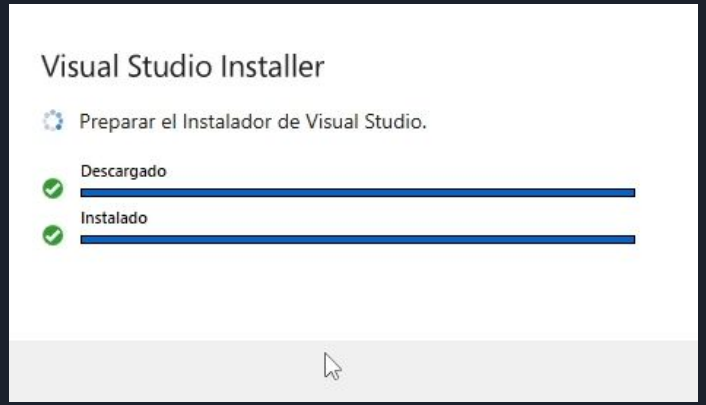
https://dotnet.microsoft.com/es-es/download

Instalación Visual Studio con .NET SDK

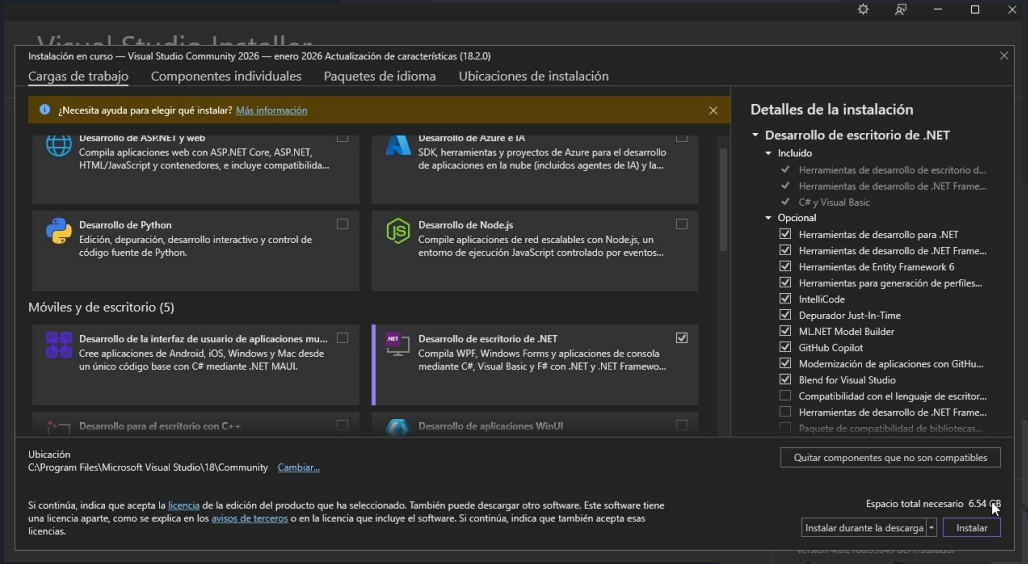
Sitio para descargar el IDE Visual Studio y el .NET SDK



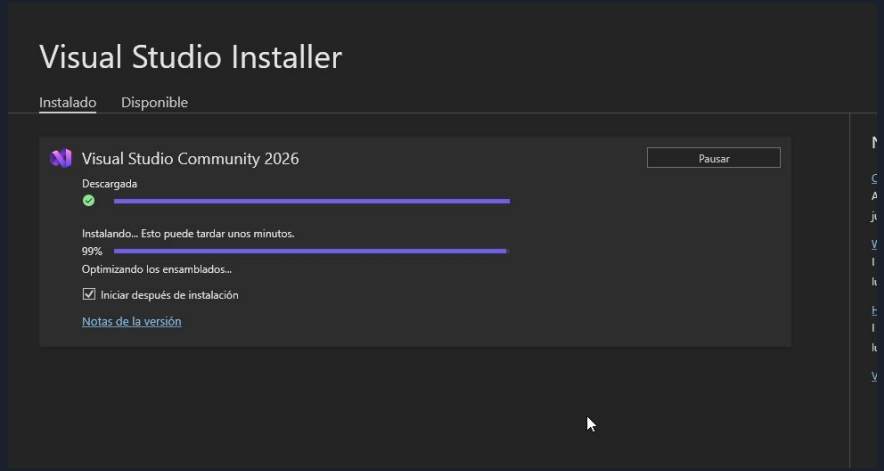
https://visualstudio.microsoft.com/es/



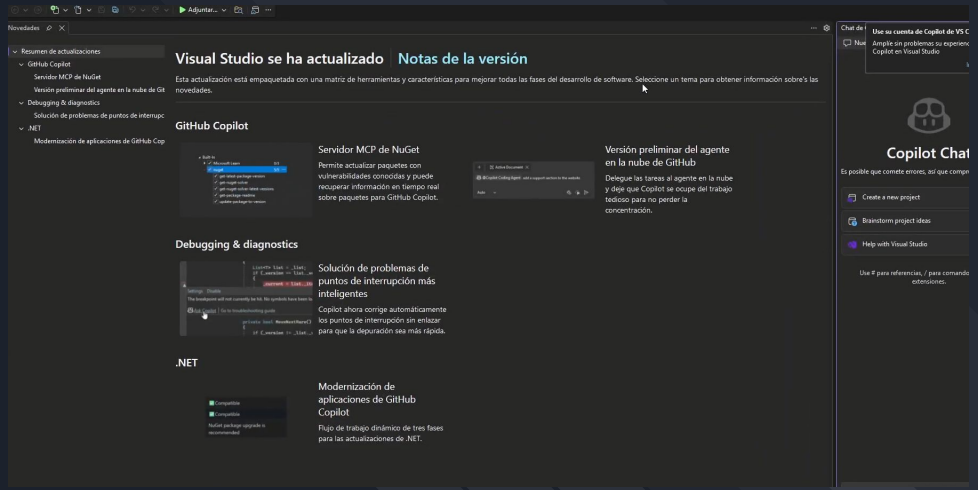
Empezar la instalacion despues de haber descargado el .exe para windows.



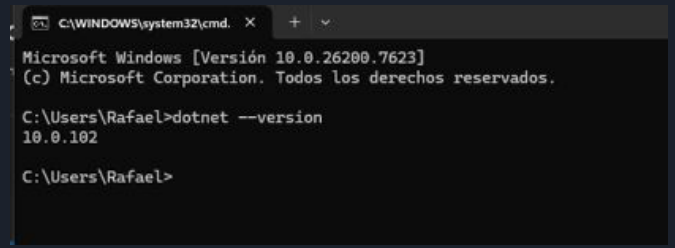
Luego se abrirá este panel donde se debe seleccionar, **Desarrollo de escritorio de .NET,** y seleccionar instalar.



Después de terminar se abrirá el IDE.



IDE instalado de forma correcta.



Si la instalación se hizo de forma correcta al probar el comando:

dotnet –version

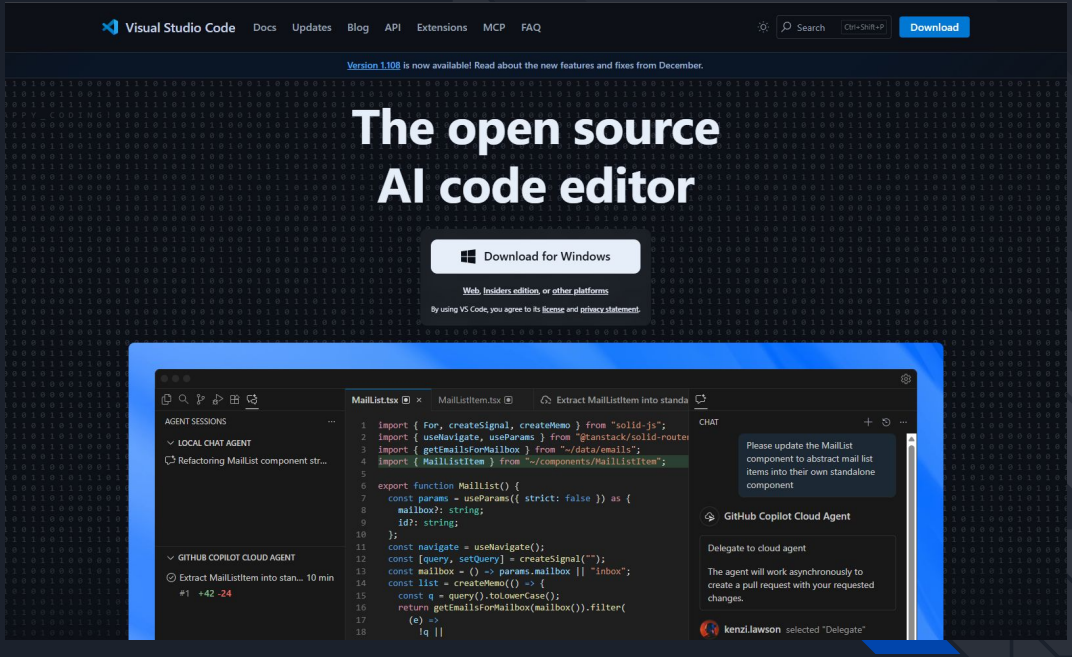
Debería de mostrar la versión instalada del SDK.

Instalación Visual Studio Code

Sitio para descargar solo el Editor de Código Visual Studio Code.

Si se usa Visual Studio Code deben de instalar la extensión de **C# Dev kit.**

Y tienen que haber instalado el **.NET SDK** para poder trabajar.

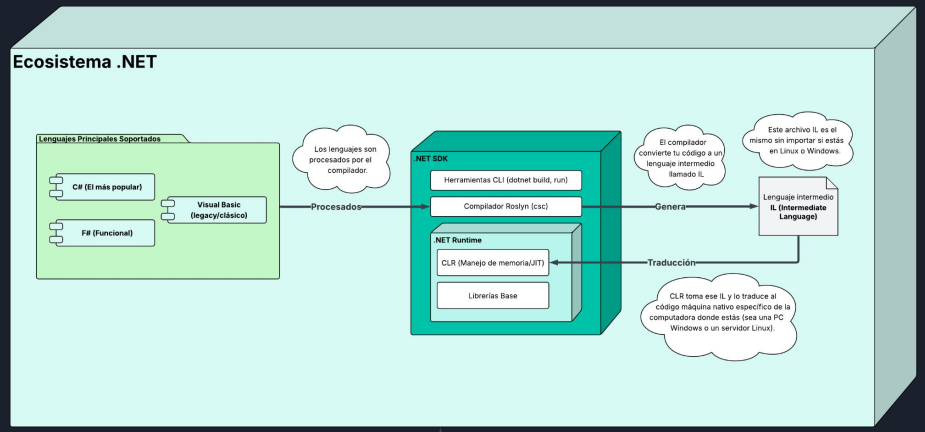


https://code.visualstudio.com/

Conceptos Clave Aprendidos

- **Diferencia Rol:** .NET es el ecosistema (plataforma) y C# es el lenguaje que utilizamos para dar las instrucciones.
- **Compilación e Independencia:** El código C# no se compila directo a máquina, sino a Lenguaje Intermedio (IL), lo que permite que corra en cualquier sistema operativo (Windows, Linux, Mac) gracias al CLR.
- **SDK vs Runtime:** Para desarrollar software es obligatorio instalar el SDK (que incluye compilador y herramientas CLI), mientras que el Runtime solo sirve para ejecutarlo.
- **Flexibilidad de Herramientas:** Podemos desarrollar tanto en editores ligeros como VS Code (ideal para rapidez y multiplataforma) como en IDEs robustos como Visual Studio (para proyectos complejos).

Entorno del Desarrollador



Desarrolladores

Valor de Esta semana

RESPONSABILIDAD

Como ingenieros, pasamos de ser usuarios a ser creadores

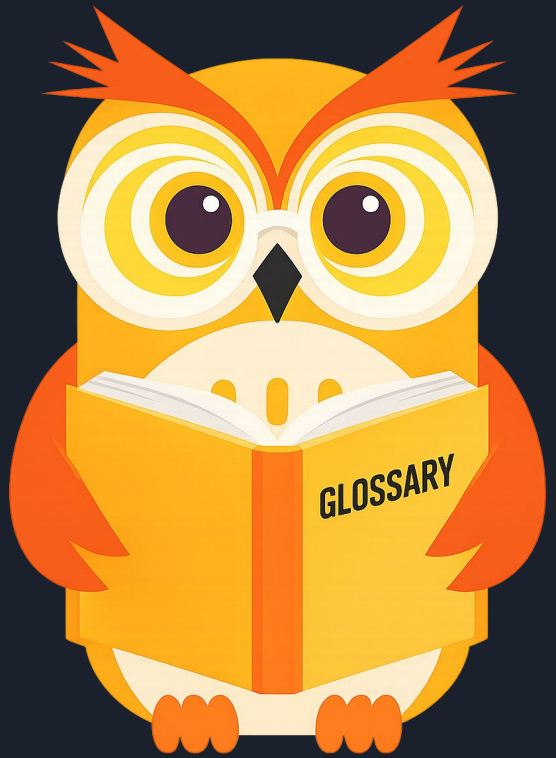
Asumimos la responsabilidad de instalar y dominar el SDK, porque nosotros construimos el software, no solo lo ejecutamos.

Referencias

- https://dotnet.microsoft.com/es-es/learn/dotnet/what-is-dotnet
- https://dotnet.microsoft.com/es-es/download
- https://learn.microsoft.com/es-mx/dotnet/core/introduction?WT.mc_id=dotnet-35129-website
- https://code.visualstudio.com/
- https://visualstudio.microsoft.com/es/

Ejemplo Práctico

Practiquemos lo Aprendido



¿Dudas?

!Gracias por su Atención¡

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA QUE TE SURJA EN LA SEMANA

ESCUELA DE INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

ECYS | DTT DESARROLLO DE TRANSFERENCIA TECNOLÓGICA

Nombre de la actividad:	Actividad Semana
Cantidad de participantes:	
Doy fe que esta actividad está planificada en dtt (Sí/No):	

Hora de inicio:	
Hora de fin:	
Duración (min):	

Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)